

Calcolatori Elettronici

5 luglio 2023

Teoria

Esercizio 1 (12 punti): Progettare l'automa a stati finiti per il controllo di un distributore automatico di bibite, secondo le seguenti regole:

- Il costo di una bibita è di 50 centesimi;
- Il distributore accetta monete da 10, 20 e 50 centesimi;
- Può essere introdotta una sola moneta per volta;
- L'uscita vale 1 se la cifra introdotta è ≥ 50 centesimi, vale 0 altrimenti;
- Quando l'uscita vale 1, l'importo introdotto viene decrementato di 50 centesimi e la bibita viene erogata;
- Il distributore non restituisce resto, ma ne consente l'utilizzo al prossimo cliente.

ATTENZIONE: viene richiesto di realizzare *soltanto* l'automa a stati finiti: non è necessario studiare le equazioni di eccitazione dei flip/flop, né realizzare un circuito equivalente.

Esercizio 2 (12 punti): Si progetti un circuito *combinatorio* che, ricevendo in ingresso un numero x di 2 bit, fornisca su un'uscita y i due bit più significativi del quadrato x^2 .

Esercizio 3 (6 punti): Spiegare le *convezioni di chiamata* del linguaggio C su sistemi x86 e discutere le motivazioni che sottostanno alla necessità del loro utilizzo per lo sviluppo di applicazioni.

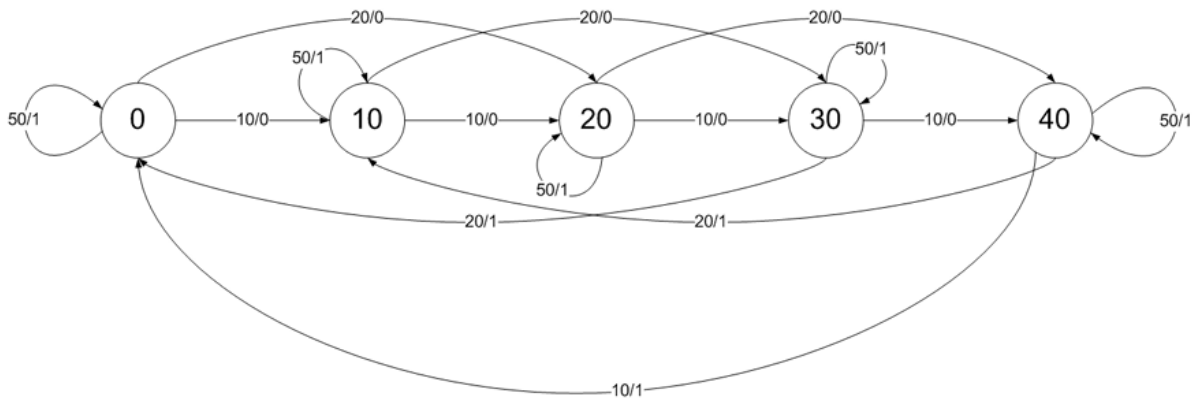
Soluzioni

Esercizio 1

Il primo passo consiste nel determinare gli alfabeti di input e di output, che possono essere i seguenti:

- $I = \{10, 20, 50\}$
- $O = \{0, 1\}$

A questo punto, è possibile realizzare l'automa a stati finiti (si è scelto di utilizzare una macchina di Mealy):



Gli stati vengono utilizzati per tenere traccia di quanti centesimi sono disponibili, considerando che, non appena vengono raggiunti almeno 50 centesimi, l'uscita assume il valore 1. Non è quindi possibile avere a disposizione più di 50 centesimi. Le transizioni vengono utilizzate per catturare le somme di centesimi. Ogni volta che una somma fornisce un risultato maggiore di 50, l'uscita varrà 1. Gli stati di destinazione rappresentano quanti centesimi sono disponibili per l'utente successivo.

Esercizio 2

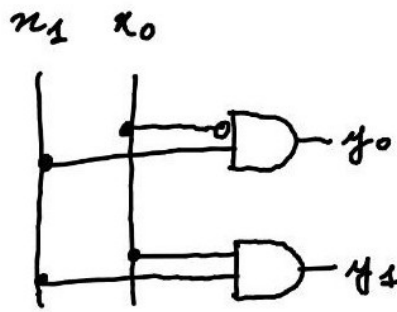
Per progettare il circuito combinatorio è necessario prima di tutto studiare la funzione booleana che deve essere implementata. La funzione può essere rappresentata mediante forma tabellare come segue:

x_1x_0	$(x)_{10}$	$(x^2)_{10}$	$(x^2)_2$	y_1y_0
00	0	0	0000	00
01	1	1	0001	00
10	2	4	0100	01
11	3	9	1001	10

Osservando la tabella, si può notare che vi è soltanto una configurazione di input per cui, rispettivamente, y_0 e y_1 assumono il valore 1. Pertanto, è possibile utilizzare direttamente i mintermini della tabella per determinare le funzioni booleane che calcolano i valori dei due bit dell'uscita:

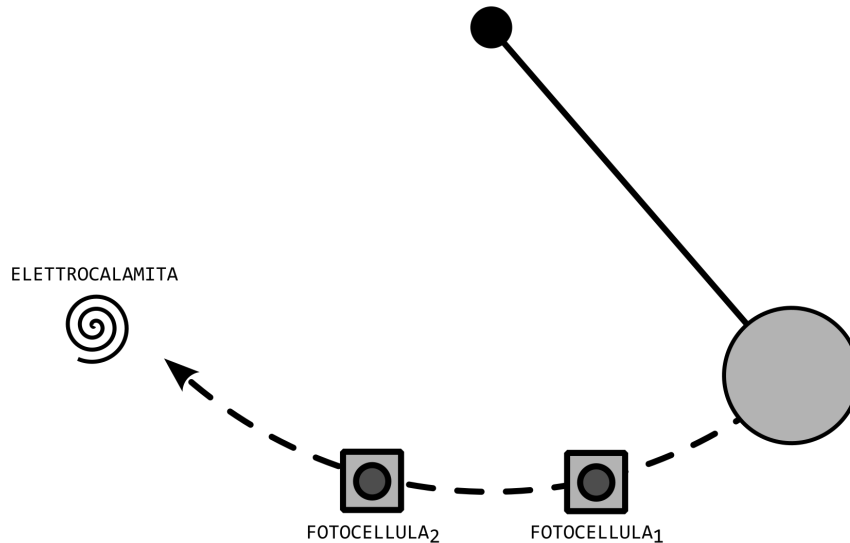
- $y_0 = x_1\overline{x_0}$
- $y_1 = x_1x_0$

Il circuito equivalente sarà:



Progetto

- Un processore `z64` controlla il sistema di scappamento elettromeccanico di un orologio a pendolo, come da figura. In tale sistema una coppia di `FOTOCELLULE` rileva il passaggio del pendolo in due posizioni ben precise, interagendo in maniera asincrona con il processore `z64`.



Il tempo intercorso tra il passaggio davanti a `FOTOCELLULA1` e `FOTOCELLULA2` viene misurato utilizzando un dispositivo sincrono `TIMER`. Tale dispositivo, una volta avviato, incomincia a misurare il tempo, alla grana dei millisecondi. Quando successivamente il processore `z64` arresta il dispositivo, il tempo trascorso può essere acquisito mediante un apposito registro di interfaccia.

Una volta determinato il tempo trascorso nel passaggio di fronte alle due `FOTOCELLULE`, lo `z64` attiva il dispositivo `ELETTROCALAMITA` per un tempo pari a $1/64$ del tempo di passaggio, al fine di fornire energia addizionale al pendolo—si può utilizzare `TIMER` anche per misurare questo tempo. Si noti che, essendo presente una sola `ELETTROCALAMITA`, non è di interesse calcolare il tempo trascorso nel passaggio tra `FOTOCELLULA2` e `FOTOCELLULA1`.

Realizzare:

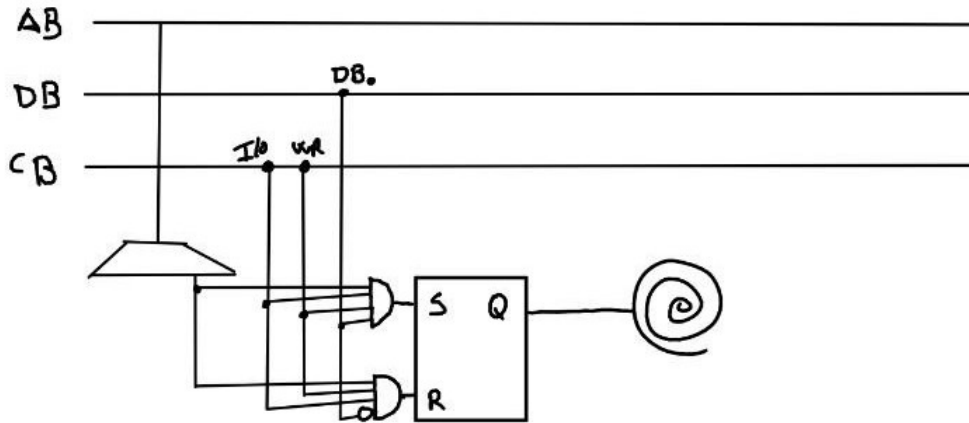
- Le interfacce delle periferiche `FOTOCELLULA`, `TIMER`, `ELETTROCALAMITA`;
- Tutto il software necessario al funzionamento del sistema, comprensivo di eventuali driver.

SUGGERIMENTO: l'avvio e l'arresto di `TIMER` possono essere interpretate come due differenti operazioni.

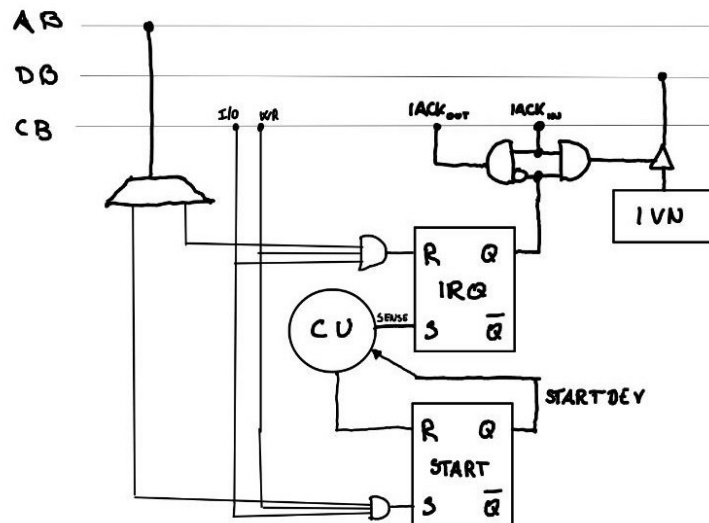
Soluzione

Si presenta una delle tantissime possibili soluzioni. Per prima cosa, progettiamo le interfacce delle periferiche.

ELETTROCALAMITA è una periferica che funziona come un interruttore. Pertanto, possiamo prevedere l'esistenza di un flip flop su cui è possibile scrivere direttamente i valori 0 e 1 per accendere/spegnere il dispositivo.



La periferica FOTOCELLULA deve poter generare richieste di interruzione. Inoltre, è necessario far sì che il dispositivo sia inibito all'invio delle interruzioni (poiché non è interessante calcolare il tempo trascorso nel passaggio tra FOTOCELLULA₂ e FOTOCELLULA₁). Per questo motivo, si può utilizzare il flip/flop START: quando il dispositivo ha percepito il passaggio del pendolo, esso solleverà la richiesta di interruzione e resetterà il flip/flop START. Il processore avvierà nuovamente il dispositivo quando lo riterrà necessario. Fino a quando la CU del dispositivo non viene esplicitamente avviata, esso non segnalerà il passaggio del pendolo.



- **TIMER** viene utilizzato per due scopi differenti. Nel caso in cui si voglia misurare il tempo in cui lasciare accesa

- Si assume che il pendolo parta nella posizione mostrata in figura. Secondo queste assunzioni, il codice può essere il

```

8      .equ TIMER_OP, 0x6
9      .equ TIMER_STATUS, 0x7
10     .equ TIMER_MILLIS, 0x8
11     .equ TIMER_READ_OP, 0 # Per facilità di lettura
12     .equ TIMER_COUNT_OP, 1
13     direzione: .byte 0 # La direzione in cui sta oscillando il pendolo, 0: verso sx; 1: verso dx
14     # Il vettore seguente è utile solo per rendere parametrica la funzione abilita_fotocellula
15     fotocellule: .word FOTOCELLULA_1_START, FOTOCELLULA_2_START
16     tempo: .long 0 # Il tempo intercorso tra fotocellula 1 e fotocellula 2
17     fotocellula2_passata: .byte 0 # Un flag per segnalare il passaggio davanti a fotocellula 2
18     .text
19     main:
20         movq $0, %rdx
21         call abilita_fotocellula
22         sti
23     .main_loop:
24         hlt
25         cmpb $1, fotocellula2_passata
26         jnz .main_loop
27         call gestisci_elettrocalamita
28         movb $0, fotocellula2_passata
29         jmp .main_loop
30
31     # Accetta come primo parametro l'id della fotocellula da avviare (0 o 1)
32     abilita_fotocellula:
33         movw fotocellule(, %rdx, 1), %dx
34         outb %al, %dx
35         ret
36
37     # Restituisce il tempo misurato da TIMER
38     acquisisci_tempo:
39         movb $TIMER_READ_OP, %dl
40         call timer_operation
41         inl $TIMER_MILLIS, %eax
42         ret
43
44     avvia_timer:
45         movb $TIMER_COUNT_OP, %dl
46         call timer_operation
47         ret
48
49     # Accetta come primo parametro l'operazione da far eseguire al timer
50     timer_operation:
51         movb %dl, %al
52         outb %al, $TIMER_OP
53         outb %al, $TIMER_STATUS
54     .bw:
55         inb $TIMER_STATUS, %al
56         btb $0, %al
57         jnc .bw
58         ret
59
60     gestisci_elettrocalamita:
61         xorl %ecx, %ecx
62         xorq %rdx, %rdx
63         movl tempo, %ebx
64         shr $6, %ebx # 1/64
65         movb $1, %al # Attiva l'elettrocalamita
66         outb %al, $ELETTROCALAMITA
67     .loop:
68         call avvia_timer
69         # Per non interrogare troppo frequentemente il timer, si aggiunge un ciclo

```

```

70     .waste_cycles:
71         addq $1, $rdx
72         cmpq $100, $rdx
73         jnz .waste_cycles
74     call acquisisci_tempo
75     addl %eax, %ecx
76     cmpl %ecx, %ebx
77     jnc .loop # Esce dal ciclo quando il tempo trascorso nel ciclo è > 1/64 * %rbx
78     movb $0, %al # Disattiva l'elettrocalamita
79     outb %al, $ELETTROCALAMITA
80     ret
81
82     .driver 0: # Fotocellula 1
83         cmpb $0, direzione
84         jnz .ret
85         call avvia_timer
86         movq $1, %rdx
87         call abilita_fotocellula
88     .ret:
89         outb %al, $FOTOCELLULA_1_IRQ
90         movb $0, direzione
91         iretq
92
93     .driver 1: # Fotocellula 2
94         outb %al, $FOTOCELLULA_2_IRQ
95         movb $1, direzione
96         call acquisisci_tempo
97         movl %eax, tempo
98         movb $1, fotocellula2_passata
99         # La fotocellula 1 viene riabilitata per consentire di determinare il cambio di direzione del
pendolo
100         movq $0, %rdx
101         call abilita_fotocellula
102         iretq

```

Prova di laboratorio

Realizzare un programma in C che accetti in input un numero n di interi, dove n è specificato dall'utente come parametro a riga di comando.

Gli n interi vengono memorizzati all'interno di un vettore. Il programma restituisce in output il numero di bit impostati a 1 nel vettore di interi inseriti dall'utente.

Soluzione

Il problema principale che viene chiesto di affrontare assume il nome di *population count* o *peso di Hamming*. Si tratta di un problema non banale e che ha molte applicazioni (ad esempio, in crittografia per stimare l'entropia di un dato o in programmi efficienti che implementano il gioco degli scacchi per conoscere la motilità dei pezzi quando si sceglie una mossa d'attacco). Tale problema è talmente rilevante che i processori x86 forniscono un'implementazione hardware di questa operazione, mediante l'istruzione assembly `popcnt`.

La soluzione più ovvia prevede di effettuare il test bit a bit:

```

1 int population_count(unsigned x)
2 {
3     int c = 0;
4     for (; x != 0; x >>= 1)
5         if (x & 1)
6             c++;
7     return c;
8 }

```

Questa implementazione, seppur corretta, ha un costo lineare nel numero di bit testati. Se il vettore inserito dall'utente è molto grande, il costo computazionale può crescere molto.

Si può allora ricorrere ad un'implementazione alternativa che si basa [sull'osservazione](#) che se si sottrae 1 dal bit meno significativo di un numero binario positivo N e si esegue l'operazione logica AND sui numeri N ed $N - 1$, il numero risultante ha un 1 in meno, essendo stato eliminato quello meno significativo. In questo modo, il costo è lineare nel numero di bit impostati ad 1 dell'input.

```

1 int population_count (unsigned x) {
2     int count = 0;
3     while (x) {
4         count++;
5         x &= x - 1;
6     }
7     return count;
8 }

```

Per completare il programma, è necessario acquisire correttamente gli interi da tastiera. Si ottiene quindi la seguente implementazione:

```

1 #include <errno.h>
2 #include <stdbool.h>
3 #include <stdio.h>
4 #include <stdlib.h>
5
6 bool get_long(long *value)
7 {
8     char buf[BUFSIZ];
9     char *endptr;
10
11     if(!fgets(buf, 1024, stdin)) {
12         return false;
13     }
14
15     errno = 0;
16     *value = strtol(buf, &endptr, 10);
17     if(errno == ERANGE || endptr == buf || (*endptr && *endptr != '\n')) {
18         return false;
19     }
20
21     return true;
22 }
23
24 long population_count(size_t size, const long array[size]) {
25     long count = 0;
26     for (int i = 0; i < size; ++i) {
27         long value = array[i];

```



```

28     while(value) {
29         count++;
30         value &= value - 1;
31     }
32 }
33 return count;
34 }
35
36 int main(int argc, char **argv)
37 {
38     long array_size;
39     long *array;
40
41     if(argc != 2) {
42         fprintf(stderr, "Usage: %s <array size>\n", argv[0]);
43         exit(EXIT_FAILURE);
44     }
45
46     // 0 is not a valid array size, so we can use it to check for invalid input
47     array_size = atol(argv[1]);
48     if(array_size <= 0) {
49         fprintf(stderr, "Invalid array size.\n");
50         exit(EXIT_FAILURE);
51     }
52
53     array = malloc(array_size * sizeof(*array));
54     printf("Enter %ld integers:\n", array_size);
55
56     for (int i = 0; i < array_size; ++i) {
57         if(!get_long(&array[i])) {
58             printf("Invalid input.\n");
59             exit(EXIT_FAILURE);
60         }
61     }
62
63     printf("The number of bits set to 1 is %ld.\n", population_count(array_size, array));
64     free(array);
65     return 0;
66 }

```